

Caso Semestral - eSports Arena Manager (Frontend)

Guía de caso para estudiantes | Desarrollo FullStack II

Asignatura	Desarrollo FullStack II - DSY1104
Tipo de trabajo	Proyecto semestral en equipos de hasta 3 integrantes
Tecnología obligatoria	EP1: HTML5, CSS3 y JavaScript. EP2: React con Bootstrap, pruebas con Jasmine y Karma. EP3: integración con microservicios Spring Boot mediante API REST y JWT.
Entregas asociadas	EP1 (30 %): base web. EP2 (30 %): framework y pruebas unitarias. EP3 (40 %): integración con backend, seguridad y sesiones.
Título del caso	Caso Semestral - eSports Arena Manager (Frontend)

1. Contexto del caso

eSports Arena Manager es una plataforma para organizar torneos de videojuegos competitivos. Administra juegos habilitados, jugadores, equipos, torneos, inscripciones, partidas, resultados, rankings, sanciones y premios, manteniendo trazabilidad de cada cambio.

Este caso cubre la capa de presentación de esa plataforma. Aquí la dificultad es que casi todo cambia de estado: un torneo pasa de abierto a en curso y luego a finalizado, una inscripción puede rechazarse por cupo o por sanción, y la tabla de posiciones se recalcula cada vez que se valida un resultado. La interfaz debe dejar claro qué acciones están disponibles y por qué otras aparecen bloqueadas. El backend de referencia es el ecosistema de microservicios de eSports Arena Manager construido en Desarrollo FullStack I.

2. Propósito formativo

El objetivo es que los estudiantes diseñen, construyan, prueben, documenten e integren la capa de presentación de eSports Arena Manager. El trabajo avanza en tres etapas acumulativas: una base web con HTML5, CSS3 y JavaScript; una migración a un framework de componentes con pruebas unitarias automatizadas; y la integración con el backend de microservicios mediante API REST, seguridad basada en tokens y control de acceso por rol.

Las tres entregas son el mismo proyecto. Cada evaluación parcial toma el producto de la anterior y lo profundiza, de modo que al cierre del semestre existe una sola aplicación frontend en un único repositorio, cuyo historial muestra esa evolución.

3. Roles de usuario y acceso a la interfaz

Rol	Qué puede hacer en la interfaz
Administrador	Configura juegos habilitados, crea torneos, define premios y reglas generales.
Organizador	Gestiona inscripciones, programa partidas, registra y valida resultados, aplica sanciones.
Jugador	Crea o integra equipos, se inscribe en torneos y consulta resultados, rankings y notificaciones.
Visitante	Consulta torneos públicos, llaves y tablas de posiciones, sin poder inscribirse.

En EP1 y EP2 los roles se simulan seleccionando un perfil de prueba. En EP3 el rol se obtiene del token JWT emitido por el servicio de autenticación.

4. Requisitos generales obligatorios del frontend

- Todas las vistas deben ser alcanzables desde un menú de navegación común y mantener una identidad visual consistente.
- El marcado debe ser semántico y válido: **header**, **nav**, **main**, **section**, **article** y **footer**, con jerarquía correcta de encabezados y atributos **alt** en imágenes.
- Los estilos se declaran en una hoja CSS externa compartida por todas las páginas. No se aceptan estilos incrustados salvo justificación técnica escrita.
- Todo formulario valida en el cliente antes de enviar, con mensajes propios mostrados junto al campo que falla.
- El equipo trabaja sobre un repositorio GitHub público con commits descriptivos y distribuidos entre los integrantes.
- En EP1 y EP2 los datos provienen de estructuras simuladas en JavaScript. En EP3 esas estructuras se reemplazan por llamadas reales a la API.
- El repositorio no debe contener credenciales, tokens ni claves. Las URL de la API se configuran por variable de entorno.
- El **README.md** indica integrantes, requisitos previos, instrucciones de ejecución y estructura de carpetas.

5. Alcance esperado para la Evaluación Parcial 1 (30 %)

En EP1 el equipo construye la base navegable de eSports Arena Manager con HTML, CSS y JavaScript, sin framework y sin backend.

- Implementar como mínimo las seis vistas indicadas en la sección de vistas requeridas, interconectadas por hipervínculos.
- Usar estructura HTML5 con elementos semánticos, imágenes con texto alternativo, botones operativos, al menos un video embebido y un footer informativo.
- Aplicar una hoja de estilos CSS externa con variables de color, tipografía definida y maquetación con Flexbox o Grid.
- Construir al menos dos formularios funcionales, incluido el formulario principal del dominio, con etiquetas asociadas mediante **label for**, atributos de autocompletado y sugerencias de ayuda.
- Validar esos formularios con JavaScript: campos obligatorios, formato de correo, rangos numéricos, longitud mínima de contraseña y coherencia entre campos relacionados. El envío se bloquea mientras existan errores.
- Cargar la información del catálogo desde arreglos JavaScript y renderizarla con manipulación del DOM, evitando listas escritas a mano en el HTML.
- Mantener el repositorio con ramas por integrante o por funcionalidad e integración mediante *merge* o *pull request*.
- Elaborar el documento ERS versión 1 con requerimientos funcionales y no funcionales, herramientas seleccionadas y propuesta de solución.

Entregables de EP1

- Enlace al repositorio GitHub público del proyecto frontend.
- Proyecto frontend comprimido.
- Documento ERS, propuesta previa, versión 1.

6. Alcance esperado para la Evaluación Parcial 2 (30 %)

En EP2 el equipo migra el proyecto de EP1 a React, lo vuelve responsivo con Bootstrap y lo cubre con pruebas unitarias. Las vistas y reglas ya definidas no cambian de alcance funcional: cambia su implementación.

- Reestructurar el proyecto en React con una organización clara de carpetas para componentes, vistas, servicios simulados y utilidades.
- Convertir las secciones repetitivas en componentes reutilizables. Cada componente recibe sus datos por propiedades y maneja su propio estado con `useState` y `useEffect`.
- Reemplazar la navegación por hipervínculos con enrutamiento de cliente, conservando las mismas rutas lógicas de EP1.
- Aplicar Bootstrap para la grilla y los componentes de interfaz, verificando el comportamiento en anchos de 360, 768 y 1280 píxeles.
- Migrar la lógica de validación a formularios controlados, manteniendo los mensajes personalizados.
- Escribir como mínimo diez pruebas unitarias con Jasmine, ejecutadas mediante Karma, según el listado obligatorio de la sección de pruebas. La rúbrica otorga el máximo con diez pruebas y baja el logro con ocho, cinco o tres, así que diez es el mínimo exigido.
- Configurar el entorno de pruebas con `karma`, `karma-jasmine`, `jasmine-core`, `karma-chrome-launcher`, `karma-webpack`, `babel-loader` con `@babel/preset-react` y `karma-coverage` para el reporte de cobertura. El montaje de componentes puede hacerse con `react-dom/client` dentro de `act()` o con una librería de testing agnóstica del ejecutor.
- Usar dobles de prueba con `jasmine.createSpy` y `spyOn` para aislar servicios, temporizadores y manejadores de eventos.
- Redactar el documento de cobertura de testing: alcance cubierto, casos diseñados, resultados obtenidos y porcentaje de cobertura por archivo.

Entregables de EP2

- Enlace al repositorio GitHub público del proyecto frontend actualizado.
- Proyecto frontend actualizado comprimido.
- Documento ERS con la propuesta terminada.
- Documento de cobertura de testing.

7. Alcance esperado para la Evaluación Parcial 3 (40 %)

En EP3 el frontend deja de usar datos simulados y se conecta al ecosistema de microservicios Spring Boot construido en Desarrollo FullStack I, a través del API Gateway.

- Implementar una capa de servicios en el frontend que centralice las llamadas HTTP, con URL base parametrizada y manejo uniforme de respuestas y errores.
- Autenticar contra el servicio de autenticación, recibir el token JWT y enviarlo en la cabecera `Authorization: Bearer` de cada petición protegida.
- Mantener la sesión activa ante recargas de página, con expiración del token, renovación cuando corresponda y cierre de sesión que limpie el estado.
- Proteger las rutas según el rol contenido en el token y ocultar o deshabilitar las acciones que el rol no puede ejecutar. La restricción visual se acompaña siempre de la validación en el backend.
- Manejar los estados de carga, vacío y error. Los códigos 400, 401, 403, 404 y 500 se traducen a mensajes que el usuario pueda entender.

- Verificar que el backend exponga sus endpoints en Swagger y que la configuración de CORS del Gateway permita el origen del frontend.
- Elaborar el manual de usuario con pantallazos detallados de los flujos principales por cada rol.
- Elaborar el documento de integración: diagrama de comunicación entre frontend, Gateway y microservicios, tabla que relacione cada vista con los endpoints que consume, recorrido del token y estrategia de manejo de errores.

Entregables de EP3

- Enlace al repositorio GitHub público del proyecto frontend.
- Enlace al repositorio GitHub público del proyecto backend.
- Proyecto frontend y proyecto backend comprimidos.
- Manual de usuario con pantallazos detallados.
- Documento de integración.

8. Vistas requeridas y descripción detallada

Las primeras seis vistas son obligatorias desde EP1. Las tres últimas se incorporan en EP3, cuando existe autenticación real.

1. Inicio

Propósito

Presentar la plataforma y destacar los torneos abiertos y en curso. **Contenido mínimo**

Encabezado con navegación, sección de torneos destacados generada desde datos simulados, bloque de próximos cierres de inscripción, video embebido con una final o presentación y footer informativo. **EP2**

Componentes `TarjetaTorneo`, `Cabecera` y `PieDePagina`. **EP3**

Consulta torneos abiertos al servicio de torneos. No requiere autenticación.

2. Listado de torneos

Propósito

Explorar todos los torneos con filtros. **Contenido mínimo**

Filtros por juego, estado y rango de fechas; buscador por nombre; tarjetas con juego, modalidad, cupos ocupados sobre cupo máximo y fecha de cierre de inscripción. **Validaciones**

El rango de fechas no acepta fecha inicial posterior a la final. Cuando ningún torneo cumple los filtros se muestra un estado vacío explícito. **EP3**

Consume torneos y juegos habilitados, resolviendo el nombre del juego a partir de su identificador.

3. Detalle de torneo

Propósito

Concentrar toda la información de un torneo. **Contenido mínimo**

Datos generales, cupos disponibles, listado de participantes inscritos, llaves o calendario de partidas por ronda con horario y estado, tabla de posiciones y premios asociados a cada posición. **EP2**

Componentes `LlaveTorneo`, `TablaRanking` y `ListaPartidas`. La ronda visible se maneja como estado local. **EP3**

Consume torneos, inscripciones, partidas, resultados y rankings, y muestra los premios asignados solo cuando el torneo está finalizado.

4. Inscripción a torneo

Propósito

Registrar la participación de un jugador o de un equipo. **Contenido mínimo**

Selección del tipo de participante, selector de equipo propio, resumen de requisitos del torneo y confirmación con el detalle de lo inscrito. **Validaciones**

No se permite inscribir fuera del plazo, ni superar el cupo máximo, ni inscribir dos veces al mismo participante en el mismo torneo, ni inscribir un equipo con menos integrantes que los exigidos por el juego. Un participante con sanción vigente recibe un mensaje que explica el bloqueo. **EP3**

Consume el servicio de inscripciones y verifica sanciones antes de habilitar el envío. Requiere sesión iniciada.

5. Gestión de equipo

Propósito

Crear y mantener un equipo. **Contenido mínimo**

Formulario de creación con nombre, juego principal y capitán; listado de integrantes con su rol dentro del equipo; acciones para agregar y quitar jugadores. **Validaciones**

Nombre de equipo obligatorio y sin duplicados en los datos disponibles, capitán obligatorio, un mismo jugador no puede repetirse en el equipo y un equipo inactivo no puede inscribirse. **EP3**

Consume los servicios de equipos y usuarios.

6. Perfil de jugador

Propósito

Mostrar la ficha competitiva de un participante. **Contenido mínimo**

Datos de contacto y apodo, equipos a los que pertenece, historial de torneos, estadísticas de victorias y derrotas, y sanciones vigentes o cumplidas. **Validaciones**

El apodo es obligatorio, no admite espacios y respeta el largo definido. **EP3**

Consume usuarios, equipos, resultados y sanciones.

7. Autenticación (EP3)

Propósito

Iniciar sesión y registrar cuentas. **Contenido mínimo**

Formulario de acceso, formulario de registro y mensajes diferenciados para credenciales inválidas y cuenta desactivada. **EP3**

Consume el servicio de autenticación, guarda el token y redirige según el rol.

8. Panel del organizador (EP3)

Propósito

Operar el torneo en curso. **Contenido mínimo**

Bandeja de inscripciones por aprobar, generación y reprogramación de partidas, registro de resultados con puntajes y ganador, y aplicación de sanciones con motivo y duración. **Validaciones**

No se registra un resultado sobre una partida inexistente o cancelada, los puntajes no admiten valores negativos y un resultado ya validado no se edita desde esta vista. **Restricción**

Ruta exclusiva de los roles Organizador y Administrador.

9. Panel de administración (EP3)

Propósito

Configuración general de la plataforma. **Contenido mínimo**

Mantenedor de juegos habilitados, creación y cierre de torneos, y definición de premios por posición. **Validaciones**

La fecha de inicio debe ser posterior al cierre de inscripción, el cupo máximo debe ser positivo y no se modifican reglas críticas de un torneo en curso. **Restricción**

Ruta exclusiva del rol Administrador. Las acciones se ocultan para el resto y el backend vuelve a validar el permiso en cada llamada.

9. Pruebas unitarias obligatorias en EP2

Mínimo diez pruebas con Jasmine ejecutadas por Karma. Se aceptan pruebas adicionales.

1. `ordenarRanking` ordena por puntos y desempata por diferencia de puntaje.
2. `calcularPuntos` asigna los puntos definidos por victoria y por derrota a partir de una lista de resultados.
3. `cuposDisponibles` resta las inscripciones vigentes al cupo máximo y nunca retorna un valor negativo.
4. `inscripcionFueraDePlazo` retorna verdadero cuando la fecha actual supera el cierre de inscripción.
5. `tieneSancionActiva` bloquea al participante con sanción vigente y permite al que ya la cumplió.
6. `equipoCompleto` valida la cantidad de integrantes exigida por el juego del torneo.
7. `TarjetaTorneo` renderiza nombre, juego y estado del torneo.
8. `TablaRanking` renderiza una fila por participante y respeta el orden recibido.
9. `FormularioInscripcion` mantiene deshabilitado el envío cuando el equipo está incompleto, verificado con un espía sobre el manejador.
10. `LlaveTorneo` muestra el texto de participante por definir cuando la partida aún no tiene rival asignado.

10. Flujos integradores mínimos

1. El administrador registra un juego habilitado y crea un torneo asociado.
2. Los jugadores inician sesión, crean o se suman a un equipo.
3. El jugador se inscribe y la interfaz valida plazo, cupo, sanciones y duplicidad antes de enviar.
4. El organizador genera las partidas entre participantes inscritos y publica horarios.
5. El organizador registra y valida los resultados de cada partida.
6. La tabla de posiciones se actualiza con los resultados validados y se refleja en el detalle del torneo.
7. Al finalizar el torneo se asignan los premios por posición y el jugador ve la notificación correspondiente.

11. Paletas de colores del caso

Las cinco paletas de este caso vienen del ecosistema competitivo: las plataformas de transmisión, los títulos que se juegan en los torneos y la señalética de arena. Son referencias de color, no una autorización para usar logos ni nombres de juegos en la interfaz.

El equipo elige una de las cinco, o propone una propia con la misma estructura de seis colores, y la declara en el ERS. La paleta se define como variables CSS en EP1 y se conserva en EP2 sobrescribiendo las variables de Bootstrap.

Tres reglas que se revisan en la corrección de EP1:

- El texto sobre su fondo debe alcanzar 4.5 a 1, y 3 a 1 en títulos, texto grande y bordes de controles. Los contrastes de estas paletas ya fueron calculados y la línea de cada una indica qué color de texto va sobre el botón primario.
- El color no puede ser el único indicador de estado. Cada aviso lleva además un icono o un texto.
- El acento se reserva para la acción principal de la pantalla y para los estados. Si todo destaca, nada destaca.

Paleta 1. Arena púrpura

Púrpura de plataforma de transmisión sobre violeta muy oscuro. El primario es el 9146FF de Twitch, con un turquesa como señal de resultado validado.

Blanco sobre el botón primario (4.6 a 1). Es el límite, así que el texto del botón no debe bajar de 16 píxeles.

Fondo	Superficie	Primario	Acento	Texto	Error
					
0E0B16	1C1430	9146FF	00F5D4	F1ECFF	FF5C7A

Paleta 2. Táctico rojo

Azul carbón con rojo de shooter táctico y texto en crema. Es la que mejor comunica el estado de partida en curso, porque el rojo se reserva para eso.

Grafito sobre el botón primario. El blanco sobre ese rojo llega a 3.4 a 1.

Fondo	Superficie	Primario	Acento	Texto	Error
					
0F1923	1B2733	FF4655	ECE8E1	ECE8E1	FF8A00

Paleta 3. Oro competitivo

Azul casi negro con dorado y turquesa, la combinación de los MOBA clásicos. El dorado funciona muy bien para podios, premios y primeros lugares de la tabla.

Grafito sobre el botón primario (7.4 a 1).

Fondo	Superficie	Primario	Acento	Texto	Error
					
010A13	0A1428	C89B3C	0AC8B9	F0E6D2	E4572E

Paleta 4. Pista y velocidad

Azul de estadio con naranja. Pensada para torneos de juegos deportivos y de conducción, donde el calendario y las llaves pesan más que el catálogo.

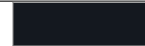
Grafito sobre el botón primario (8.7 a 1).

Fondo	Superficie	Primario	Acento	Texto	Error
					
0A1A2F	12294A	F79B14	00A8E8	F2F7FB	FF5A5F

Paleta 5. Podio claro

Clara, azul marino con rojo. Es la opción para el panel del organizador, que suele usarse con luz encima y en pantallas compartidas.

Blanco sobre el botón primario. El rojo de acento solo como relleno con texto oscuro.

Fondo	Superficie	Primario	Acento	Texto	Error
					
FAFAFA	FFFFFF	1D3557	E63946	14181F	B42318

Cómo declararla en el CSS

La hoja externa de EP1 define los colores una sola vez y el resto de las reglas los referencia. El ejemplo usa la paleta 1.

```
:root {
  --color-fondo:      #0E0B16;
  --color-superficie: #1C1430;
  --color-primario:   #9146FF;
  --color-acento:     #00F5D4;
  --color-texto:      #F1ECFF;
  --color-error:      #FF5C7A;
  --radio-borde: 8px;
}

body { background: var(--color-fondo); color: var(--color-texto); }
.tarjeta { background: var(--color-superficie); border-radius: var(--radio-borde); }
.boton-primario { background: var(--color-primario); }
.mensaje-error { color: var(--color-error); }
```

En EP2 esas mismas variables alimentan los estilos de Bootstrap, ya sea sobrescribiendo las variables de la versión 5 o aplicando clases propias que las leen.

12. Sugerencias para la defensa técnica individual

La presentación pesa 60 % en cada evaluación parcial y se califica de forma individual. Cada estudiante debe poder explicar el trabajo del equipo completo, no solo la parte que escribió.

- Explicar la estructura semántica de una vista y justificar la elección de cada etiqueta.
- Mostrar dónde está definido un estilo en la hoja externa y cómo se aplica en más de una página.
- Agregar en vivo una validación a un formulario y demostrar el mensaje de error resultante.
- Explicar un componente: qué recibe por propiedades, qué guarda en su estado y cuándo se vuelve a renderizar.
- Ejecutar la suite de pruebas, explicar un caso y mostrar el uso de un espía o simulación.
- Recorrer el camino del token desde el inicio de sesión hasta una petición protegida.
- Provocar una respuesta 401 o 403 y explicar cómo reacciona la interfaz.
- Mostrar el historial del repositorio y justificar la distribución de tareas entre integrantes.

13. Evidencias mínimas por entrega

Entrega	Evidencias
EP1 (30 %)	Repositorio GitHub público, proyecto comprimido, ERS versión 1, capturas de las vistas y de las validaciones en funcionamiento.
EP2 (30 %)	Repositorio actualizado, proyecto comprimido, ERS terminado, documento de cobertura de testing, salida de la ejecución de la suite de pruebas.
EP3 (40 %)	Repositorio frontend y backend, ambos proyectos comprimidos, manual de usuario con pantallazos por rol, documento de integración, captura de Swagger con los endpoints consumidos.

14. Mapa de indicadores por entrega

Las ponderaciones corresponden a la pauta oficial de cada evaluación parcial. La columna de evidencia indica dónde se demuestra el logro.

Evaluación Parcial 1

Código	Qué evidencia el equipo	Pond.
<i>Situación evaluativa 1: encargo, evaluación grupal (40 %)</i>		
IE1.1.1	Contenido web con estructura y etiquetado HTML actual, navegación por hipervínculos, imágenes, botones, video, elementos de navegación, formularios y footer.	8 %
IE1.1.2	Hoja de estilos CSS personalizada añadida de forma externa a todas las páginas.	10 %
IE1.2.1	Validaciones en JavaScript con sugerencias y mensajes de error personalizados en los formularios.	10 %
IE1.3.1	Cambios en el repositorio remoto, comentados de forma coherente y con tareas distribuidas.	12 %
<i>Situación evaluativa 2: presentación, desempeño individual (60 %)</i>		
IE1.1.3	Explicación de la creación del contenido web y de la importancia de la semántica correcta.	10 %
IE1.1.4	Descripción del uso de hojas de estilo externas para una interfaz atractiva y mantenible.	15 %
IE1.2.2	Demostración de las validaciones en JavaScript y su aporte a la experiencia de usuario.	15 %
IE1.3.2	Justificación del trabajo colaborativo en el repositorio remoto.	20 %

Evaluación Parcial 2

Código	Qué evidencia el equipo	Pond.
<i>Situación evaluativa 1: encargo, evaluación grupal (40 %)</i>		
IE2.1.1	Proyecto diseñado con framework de JavaScript, estructura sólida y funcionalidades completas.	10 %
IE2.1.2	Componentes React con gestión de propiedades y estados, más diseño responsivo con Bootstrap.	10 %
IE2.2.1	Diez pruebas unitarias con Jasmine y Karma que verifican lógica y comportamiento de los componentes.	12 %
IE2.3.1	Proceso de testeo: entorno configurado, uso de mocks, análisis de resultados y cobertura.	8 %
<i>Situación evaluativa 2: presentación, desempeño individual (60 %)</i>		
IE2.1.3	Exposición del diseño del proyecto con framework de JavaScript.	15 %
IE2.1.4	Demostración del desarrollo de componentes, propiedades, estados y responsividad.	20 %
IE2.2.2	Demostración de las diez pruebas unitarias y su aporte a la calidad del código.	15 %
IE2.3.2	Explicación del proceso de testeo unitario y de los conceptos aplicados.	10 %

Evaluación Parcial 3

Los indicadores IE3.1.1 e IE3.2.1 se evidencian con el backend integrado. El equipo puede presentar el ecosistema de microservicios desarrollado en Desarrollo FullStack I, siempre que esté conectado a base de datos, exponga los endpoints que el frontend consume y los documente en Swagger.

Código	Qué evidencia el equipo	Pond.
<i>Situación evaluativa 1: encargo, evaluación grupal (40 %)</i>		
IE3.1.1	Backend conectado a base de datos, con lógica de negocio y modelo de datos acordes al caso.	8 %
IE3.2.1	Servicios API REST en Spring Boot con operaciones CRUD documentadas en Swagger.	8 %
IE3.2.2	Integración efectiva entre frontend y backend mediante la API REST.	6 %
IE3.3.1	Autenticación con roles y tokens JWT en el backend.	6 %
IE3.3.2	Gestión de sesiones en el frontend con persistencia ante recargas.	6 %
IE3.3.3	Restricciones de acceso a funcionalidades en el frontend según permisos.	6 %
<i>Situación evaluativa 2: presentación, desempeño individual (60 %)</i>		
IE3.1.2	Descripción del backend conectado a base de datos y de su modelo de datos.	12 %
IE3.2.3	Explicación de los servicios API REST y de su documentación en Swagger.	12 %
IE3.2.4	Justificación de la integración y análisis del flujo de datos.	10 %
IE3.3.4	Descripción de la autenticación con roles y JWT.	10 %
IE3.3.5	Exposición del sistema de gestión de sesiones en el frontend.	8 %
IE3.3.6	Explicación de las restricciones de acceso, con ejemplos de interfaces restringidas.	8 %